

PROJECT DESIGN PHASE

Problem – Solution Fit | Implement Client Script & UI Policy (Incident)

Date	Team ID	Project Name	Maximum Marks
01 October 2026	01 October 2026	Implement Client Script & UI Policy (Incident)	2 Marks

Problem–Solution Fit Statement: The project addresses inconsistent Incident data entry by using UI Policies and Client Scripts to dynamically control form behavior, auto-populate values, validate submissions, and prevent incomplete records.

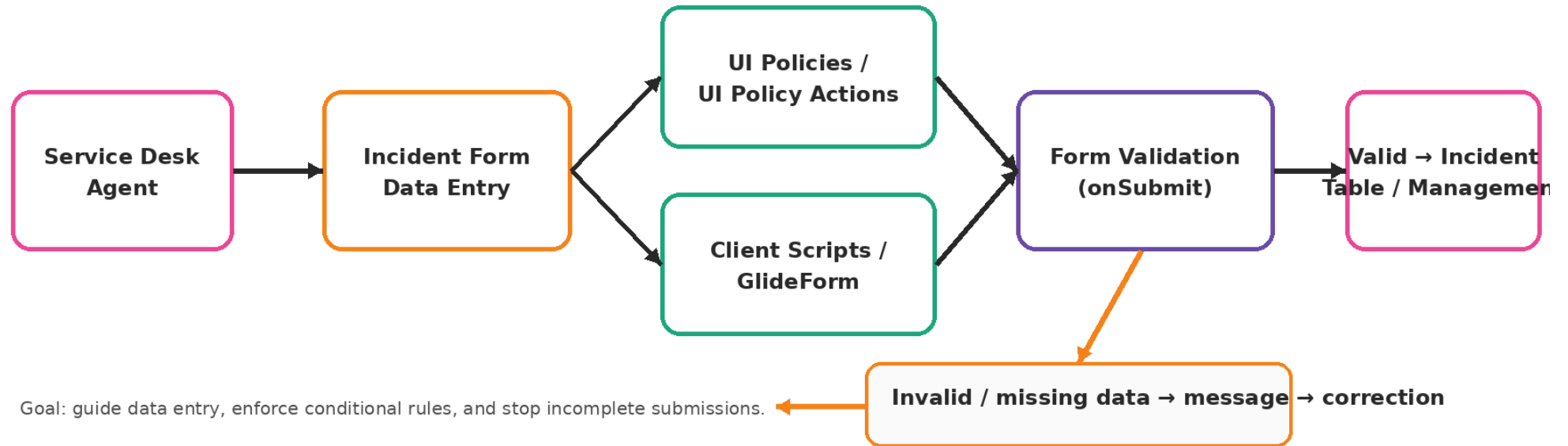
Problem-Solution Fit Canvas

Purpose / Vision: Ensure complete, consistent and valid Incident records

CUSTOMER	1. CUSTOMER SEGMENT(S) - Primary: Service Desk Agents / Incident Users - Secondary: IT Support Staff and Incident Administrators - Users who create, update, triage, and maintain Incident records in ServiceNow.	6. CUSTOMER CONSTRAINTS - Users have different levels of ServiceNow knowledge. - Incident creation must be quick during support operations. - Rules depend on correct ServiceNow configuration. - Client-side controls must be carefully designed to avoid confusing or unnecessary validation. - Roles and ACLs still govern security; client-side controls alone are not a security boundary.	5. AVAILABLE SOLUTIONS - Manual checking by Service Desk users. - Static mandatory fields and standard form configuration. - User guidance, SOPs, and training. - Existing UI Policies / Client Scripts where configured. - Limitation: manual checks can be inconsistent; static rules cannot cover every conditional case.	FUNCTIONALITY
	2. JOBS-TO-BE-DONE / PROBLEMS - Create Incident records with complete and accurate information. - Know which fields become mandatory or optional based on selected values. - Prevent incomplete or invalid Incident submissions. - Maintain consistent Incident data for triage, routing, reporting, and SLA tracking.	9. PROBLEM ROOT CAUSE - Incident data quality can depend too heavily on user awareness and manual checking. - Conditional business rules may not be enforced consistently at the form level. - Without dynamic field control and save-time validation, incomplete or inconsistent records can be submitted.	7. BEHAVIOUR - Open the Incident form and enter basic details. - Select or change values such as category, impact, urgency, or priority. - Respond to fields that become mandatory or change visibility/state. - Review the form and submit the Incident. - Correct validation errors and resubmit when required.	
TRIGGER	3. TRIGGERS - A user opens the New Incident form. - A category, priority, impact, urgency, or other controlling value is changed. - A condition requires a field to become mandatory or change its behavior.	10. YOUR SOLUTION - Implement UI Policies and UI Policy Actions to dynamically control field visibility, mandatory state, and related form behavior. - Implement Client Scripts using GlideForm for dynamic behavior and auto-population. - Use an onSubmit Client Script for final client-side validation and prevent submission when required conditions are not met. - Result: more complete, consistent, and valid Incident records before they enter Incident Management workflows.	8.1 ONLINE CHANNELS ONLINE: ServiceNow Incident form, dynamic field behavior, validation messages, and Incident list/table.	BENEFIT
	4. EMOTIONS: BEFORE / AFTER - BEFORE: Uncertain, frustrated, or delayed when rules are unclear or validation is manual. - AFTER: More confident and in control because the form clearly guides data entry and prevents invalid submission.		8.2 OFFLINE CHANNELS OFFLINE: Service desk procedures, team guidance, training, and support communication.	

Detailed Solution Flow Diagram

Solution Flow — Client Script & UI Policy (Incident)



Canvas Content — Project Specific

1. CUSTOMER SEGMENT(S)

- Primary: Service Desk Agents / Incident Users
- Secondary: IT Support Staff and Incident Administrators
- Users who create, update, triage, and maintain Incident records in ServiceNow.

2. JOBS-TO-BE-DONE / PROBLEMS

- Create Incident records with complete and accurate information.
- Know which fields become mandatory or optional based on selected values.
- Prevent incomplete or invalid Incident submissions.
- Maintain consistent Incident data for triage, routing, reporting, and SLA tracking.

3. TRIGGERS

- A user opens the New Incident form.
- A category, priority, impact, urgency, or other controlling value is changed.
- A condition requires a field to become mandatory or change its behavior.
- The user attempts to submit an Incident.

4. EMOTIONS: BEFORE / AFTER

- BEFORE: Uncertain, frustrated, or delayed when rules are unclear or validation is manual.
- AFTER: More confident and in control because the form clearly guides data entry and prevents invalid submission.

5. AVAILABLE SOLUTIONS

- Manual checking by Service Desk users.
- Static mandatory fields and standard form configuration.
- User guidance, SOPs, and training.
- Existing UI Policies / Client Scripts where configured.
- Limitation: manual checks can be inconsistent; static rules cannot cover every conditional case.

6. CUSTOMER CONSTRAINTS

- Users have different levels of ServiceNow knowledge.
- Incident creation must be quick during support operations.
- Rules depend on correct ServiceNow configuration.
- Client-side controls must be carefully designed to avoid confusing or unnecessary validation.
- Roles and ACLs still govern security; client-side controls alone are not a security boundary.

7. BEHAVIOUR

- Open the Incident form and enter basic details.
- Select or change values such as category, impact, urgency, or priority.
- Respond to fields that become mandatory or change visibility/state.
- Review the form and submit the Incident.
- Correct validation errors and resubmit when required.

8. CHANNELS OF BEHAVIOUR

- ONLINE: ServiceNow Incident form, dynamic field behavior, validation messages, and Incident list/table.
- OFFLINE: Service desk procedures, team guidance, training, and support communication.

9. PROBLEM ROOT CAUSE

- Incident data quality can depend too heavily on user awareness and manual checking.

- Conditional business rules may not be enforced consistently at the form level.
- Without dynamic field control and save-time validation, incomplete or inconsistent records can be submitted.

10. YOUR SOLUTION

- Implement UI Policies and UI Policy Actions to dynamically control field visibility, mandatory state, and related form behavior.
- Implement Client Scripts using GlideForm for dynamic behavior and auto-population.
- Use an onSubmit Client Script for final client-side validation and prevent submission when required conditions are not met.
- Result: more complete, consistent, and valid Incident records before they enter Incident Management workflows.

References

1. Problem–Solution Fit Canvas, Daria Nepriakhina / Amaltama.com (template source), Creative Commons Attribution-NonCommercial-NoDerivatives 4.0.
2. Idea Hackers Network — Problem-Solution Fit Canvas: <https://www.ideahackers.network/problem-solution-fit-canvas/>
3. Epicantus — Problem-Solution Fit Canvas: <https://medium.com/@epicantus/problem-solution-fit-canvas-aa3dd59cb4fe>

Project alignment note: This Phase III document follows the uploaded mentor template’s 10 canvas areas and purpose/vision structure. ☞filecite references are included in the chat response; the document itself preserves the source URLs and attribution.